Naive GEMM on Big Red 200: Loop Order, Layout, and L1 Misses

Short Report

Pradyot Bathuri
Indiana University

April 2026

Artifacts: `bigred200_results/results.csv`, `mm_papi.c`; cluster: **Big Red 200** (Slurm).

Purpose and scope

This note summarizes a completed measurement campaign (proposal → executed experiments → CSV and plots → draft long-form write-up). We study **naive** single-precision $C \mathrel{+}= AB$ with **six** compile-time loop permutations $(ijk, \ldots, kji)$ and **two** layouts (**row-major** / **column-major**). PAPI records **L1 data misses** and elapsed time in the **multiply** region only; GFLOPS uses $2N^3$ operations for square $N \times N$ matrices.

Method (compressed)

Item	Setting
Kernel	Triple loop GEMM, `float`, square $M = N = K$
Variants	2 layouts × 6 orders ⇒ **12** binaries (one order + layout per build)
Instrumentation	PAPI high-level region around **multiply** (init excluded)
Platform	IU **Big Red 200**, batch jobs via **Slurm**
Sweep	N from small values through **5000** (multiple points per decade)

Caveats (unchanged from full paper): L1-only counters; naive code ≠ BLAS; at small N the compiler may erase nominal source-level differences.

Results with numbers

Small N — no separation yet. At $N = 10$, every order records ∼**22** L1D misses (both layouts). At $N = 100$, **all six orders** share the **same** logged values: **17 642** L1 misses, **2.7895 GFLOPS**, identical seconds—so loop order **cannot be distinguished** at this scale with these metrics.

Large $N = 5000$ — strong separation. Head-to-head pairs (same layout; orders that remain the practical comparison at this size):
Rounded interpretation: the faster order runs ∼**38%** higher GFLOPS and ∼**25%** fewer L1D misses than the slower partner in each pair; wall time drops ∼**27–37%** for the same arithmetic work.

Layout	Fast	GFLOPS	L1D misses	Time (s)	Slow	GFLOPS	L1D misses	Time (s)
Row-major	**ikj**	**6.121**	1.189×10^{10}	**40.845**	kij	4.449	1.576×10^{10}	56.198
Col.-major	**jki**	**6.119**	1.189×10^{10}	**40.858**	kji	4.482	1.577×10^{10}	55.781

Row-major (`ikj` vs `kij`) and column-major (`jki` vs `kji`) show **nearly identical** GFLOPS and miss totals for the "fast" and "slow" roles—evidence that **layout and order must be analyzed jointly**, not as a single universal "best" nest.

Mid-range behavior. Plots and bar charts (e.g. $N = 500$ row-major) show **large spreads** across orders—four nests at high L1 counts versus `kij`/`kji` at much lower misses—consistent with inner-loop stride mattering once the working set no longer sits trivially in L1. GFLOPS vs. N is **non-monotonic** (peaks and "cliffs"), as expected for unblocked matmul crossing cache boundaries.

Progress check

Milestone	Status
Kernel + PAPI + layout/order matrix	**Done**
Slurm runs on Big Red 200 + `results.csv`	**Done**
Figures (GFLOPS vs. N, L1 bars at selected N)	**Done**
Long-form draft (`Matmul_Loop_Order_PAPI_BigRed200_Results.md`)	**Done**
Exact node CPU model, compiler line, Slurm excerpt in prose	**Todo** (reproducibility)
Replicated runs (median / spread)	**Optional next**
L2/LLC counters or roofline / tiled kernel	**Future extension**

Bottom line: The project is **on track** for a results-driven HPC narrative: quantitative **large-N** effects are clear and internally consistent (misses $\leftrightarrow$ time $\leftrightarrow$ GFLOPS), and **small-N degeneracy** is documented rather than hidden. Remaining work is **polish and optional strengthening** (repeats, extra counters, blocked baseline), not recovery from missing data.

References (short)

PAPI (hardware counters); Williams et al. (roofline, for future work); Dongarra et al. (BLAS context). See full manuscript for complete list.